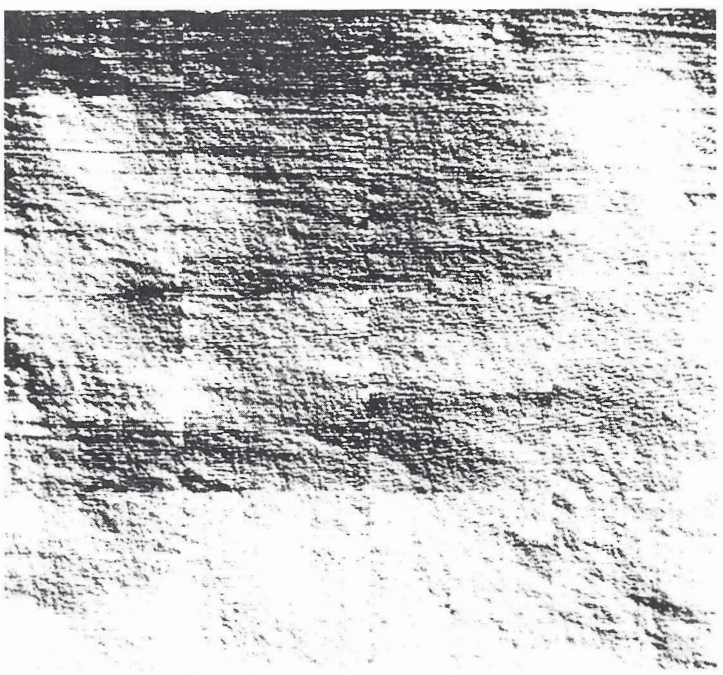
MULTIPLE AP LANDSAT ADAPTIVE FILTERING APPLICATIONS FOR THE FPS-5000 SERIES

Photos: Frank Jones

Original LANDSAT image of Canning Basin, western Australia.



Adaptive filtered version.

Figure 1.

INTRODUCTION

LANDSAT, one of a series of earth resources imaging satellites, provides the tools necessary to allow efficient exploratory investigations of remote areas of the world. Data transmitted from these satellites covers approximately 110 km^2, the processing of which easily taxes the capabilities of traditional computer systems. LANDSAT images are useful to geophysicists, who extract specific kinds of information by filtering the data, thus revealing or highlighting certain features and subduing of suppressing others.

The images shown in Figure 1 represent portions of desert areas in the Canning Basin of western Australia. A typical application of filtering suppresses the sand dune ridge formations (seen as predominant diagonal stripes) to reveal underlying geologic structures and unique features. This type of filtering requires special processing techniques, such as those using the FPS-5000 Series array processor. This article briefly describes the hardware and algorithms used in such an application.

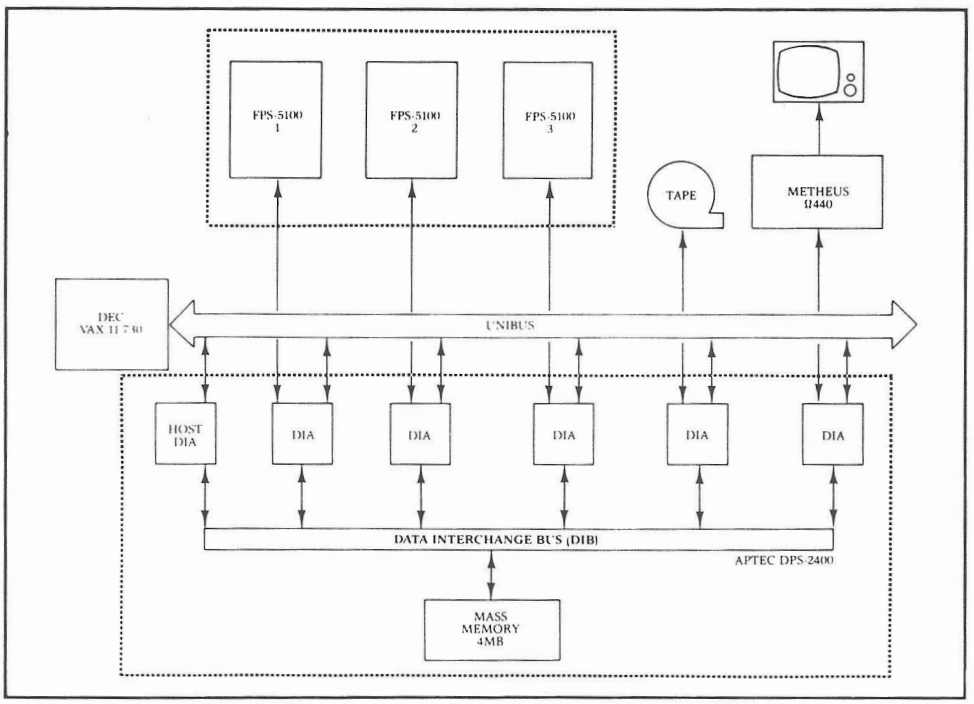


Figure 2. LANDSAT Adaptive Filter Diagram (process repeated for every 128 x 128 subimage)

System Hardware

The system hardware used in this application consists of a DEC VAX 11/730 host computer, three prototype FPS-5100 array processors, an Aptec DPS-2400 dimensional processing system, a Systems Industries 9700 magnetic tape system, and a Metheus Omega 440 display system (see Figure 2).

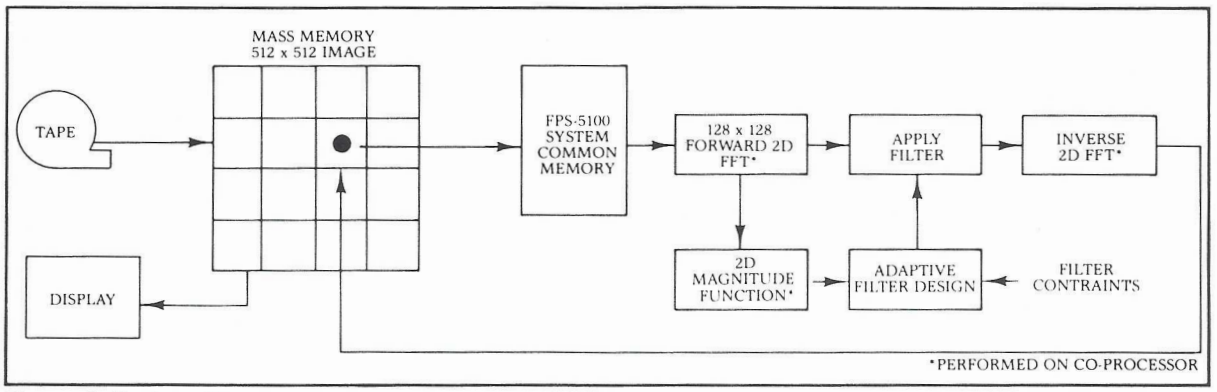


Figure 3. System Block Diagram.

The Data Interchange Adapters (DIAs) interconnect the various system elements and the VAX host system via a Data Interchange Bus (DIB). This scheme permits high-speed data transfers between each device and the four-megabyte mass memory in the DPS-2400. In operation the VAX performs the overall system control, the FPS-5100 array processors provide the processing power required to achieve the desired speed and accuracy, and the Metheus display system displays the resulting images. The tape subsystem serves as the input device to this system.

Algorithm Description

To achieve the results described above, the algorithm makes use of adaptive filtering techniques, Fast Fourier Transforms (FFTs), and sub-image management techniques. During processing the data is stored in the DPS mass memory, processed in parallel by the FPS-5100 array processors, and finally displayed on the high-resolution monitor.

Processing

Initially, the source image is moved directly from magnetic tape to the DPS-2400 mass memory by a single host system command (see Figure 3). This data, representing a 512-by-512 pixel image, is first divided into 16 windows, each containing an array of 128-by-128 pixels. These arrays are then made available

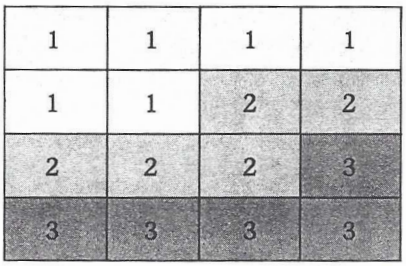


Figure 4. Assignment of Sub-image areas to three FPS-5100 array processors.

to each of three FPS-5100 array processors (see Figure 4). Each array processor reads one of the 128-by-128 arrays into its local system common memory in preparation for the filtering operation. Double buffers are maintained within each array processor so that processing and DIA data transfers may be overlapped. Within an array processor, the processing task is further divided between the control processor and the internal co-processor. As processing of each window is completed, the results are returned to the mass memory and then transferred to the display for viewing of the partially processed image. As one array processor finishes with its window, another is transferred to it until all 16 have been processed and displayed.

Within an individual array processor, the processing begins by performing a two-dimensional real-to-complex FFT on the window. A magnitude image is built representing the spatial spectrum of the data, and is used to generate a two-dimensional filter function which eliminates the dominant sand dune structure.

The FPS-5100 prototypes each contain an XP22 arithmetic co-processor which is responsible for executing a forward two-dimensional FFT, computing the magnitude function, and executing the inverse two-dimensional FFT. The control processor maintains the buffer pointers for mass memory and designs and applies the adaptive filter.

The two-dimensional filter operates by selecting a sub-region within the spatial magnitude array. Both the lowest and highest frequency corners of the sub-region are preserved and a threshold value is compared to magnitude values in the central area. Wherever the threshold is exceeded, the spatial energy in the image is zeroed.

At this point an inverse two-dimensional FFT reconstructs the image window with the stripe (or ridge) details removed. This final sub-image is then scaled to byte-range values and transferred back to the mass memory. It is then moved to the Metheus display system for viewing by the user.

The combined processing time for all I/O transfers, processing of the 16 sub-images, storage of results in mass memory, and display of the completed image is shown in the table below.

PROCESSING TIME
PERFORMANCE SUMMARY

AP Type	No. APs	Time/sec.
FPS-5105	1	7.43
	2	3.95
FPS-5100p	1	4.56
	2	2.52
	3	2.13

Summary

This application shows the efficient use of a high-performance computing system with multiple array processors having integrated distributed I/O control. The techniques used can easily be applied to a wide class of image processing tasks, including histogram equalization, principal component analysis, and texture enhancement operators.

The author acknowledges Terry Harran (Floating Point Systems) for designing the two dimensional FFT co-processor programs; Gerry Feldcamp (Aptec) for developing DIA programs; and Rex Tracy (Floating Point Systems) for inspiration in designing the adaptive filters.

Ian Curington is a Product Specialist with Floating Point Systems, Inc., where his responsibilities include technical marketing of I/O products, graphics, and image processing applications for 38-bit products. Before assuming his present position. Mr. Curington was a Systems Engineer in the FPS Performance Analysis group. Prior to joining FPS in 1979, he worked as a Systems Programmer on real-time data acquisition and control systems. Mr. Curington holds a BS degree in mathematics from Lewis and Clark College. His professional interests include computer graphic architectures, digital image synthesis. and motion dynamics. (800) 547-1445, extension 783.

NEW 38-BIT RELEASES

FPS-100 D04-000 RELEASE NOTES FOR PERKIN-ELMER (PE01)

1 DEVIATIONS FROM STANDARD RELEASE

Overlapped DMA with host and AP execution are supported. Because of the hardware characteristics of the array processor interface board, it is not possible to communicate with the AP while a DMA is in progress. However, a DMA can be started while the AP is running.

Because of the characteristics of host operating system OS/32, the routines SREAD and TERM are not implemented.

Supervisors are not supported on this host.

Chain mode is not supported on this host.

2 RELEASE ENHANCEMENTS

The following enhancements have been added for FPS-100 D04-000 Release:

- LOD100 Table Sizes – The default table sizes are host dependent. The tables are larger for those hosts that typically have more memory space available. LOD100 can handle more than 128 external symbols on machines with sufficient memory space.

  The table sizes for this host are:

200	External Symbols
200	Subroutine Names
200	Common Block Names

- LOD100 Bug Fixes – The /SIZE option on the OUTPUT command now works.
- IOCAL Table Sizes – IOCAL has been modified to handle larger program sizes. The default table sizes are now host dependent. The tables are larger for those hosts that typically have more memory space available.

  Table sizes for this host are:

512	Maximum IOCAL program size (AP words)
200	Maximum number of symbols

- IOCAL Usage with LOD100 – The object output is now compatible with LOD100. $SUB in IOCAL produces both PS and MD external symbols. The PS size is zero. This feature allows extraction of IOCAL code from libraries without requiring the FORCE statement in LOD100. In addition, IOCAL accepts a $EXT pseudo-op. The named symbol must be an MD symbol such as IOCAL code or MD Common. This allows an IOCAL program to call other IOCAL programs that reside in a library file.
- AP-FORTRAN MD Addressing – MD addressing beyond 32K in machines with Page Select now works properly in all cases. The maximum size for a single array is limited to less than 32K.
- AP-FORTRAN Common Block Transfer – An APLOCAL statement has been added to specify common blocks that are used only within the AP. The user can now specify all combinations for common block movement to or from the AP (both ways, to the AP, from the AP, neither way).